

compare them to find leaks or differences.

The pros are:

- No external dependencies.
- Precise for Python-allocated memory (lower overhead than some tools).
- Excellent for debugging leaks; shows exactly where allocations happen.
- Supports peak/current traced memory and object-specific tracebacks.

Cons:

- Only tracks Python allocations (misses C extensions like NumPy unless they use Python's allocator).
- Requires starting tracing early (ideally at program start).
- More low-level; needs code to take/compute snapshots.

When to use: Use it when investigating memory leaks, finding allocation hotspots, or comparing memory before/after code sections. It's great for detailed, programmatic analysis in production-like debugging.

```
import tracemalloc

tracemalloc.start()

# create huge list
a = [i for i in range(5_000_000)]

current, peak = tracemalloc.get_traced_memory()
print(f"Current memory: {current / 10**6:.2f} MB; Peak: {peak / 10**6:.2f} MB")

tracemalloc.stop()

##OUTPUT
% /usr/bin/python3 tracemalloc_sample.py
Current memory: 180.21 MB; Peak: 180.21 MB
```

Using *memory_profiler*

This is a third-party package (pip install *memory_profiler*) for line-by-line memory profiling.

What it measures: Measures memory usage (typically Resident Set Size -- the portion of a process that is living in RAM right now) after each line in decorated functions, showing increments. Uses *psutil* as the default backend (optional switch to *tracemalloc* for more precise Python-only allocations).

The pros are:

- Easy line-by-line view (similar to *line_profiler* for time).
- Works cross-platform (Windows/Linux/Mac).
- Plots memory over time with *mprof*.

- Backend flexible (can switch between *psutil* and *tracemalloc*).

Cons:

- External dependency (and often needs *psutil*).
- Higher overhead due to frequent sampling of process memory.
- RSS is coarse and may overestimate memory (includes shared libs, OS allocations).
- Less precise than *tracemalloc* for pinpointing pure Python object allocations.

When to use: Good for quick, high-level overview of memory growth per line in functions/scripts. Ideal for exploratory profiling for scripts or functions where you only need to know which lines are causing memory jumps, not deep allocation traces.

```
from memory_profiler import memory_usage

def measure_ram(func):
    def wrapper(*args, **kwargs):
        mem_before = memory_usage()[0]
        result = func(*args, **kwargs)
        mem_after = memory_usage()[0]
        print(f"RAM used: {mem_after - mem_before:.2f} MB")
        return result
    return wrapper

@measure_ram
def sample_func():
    a = [i for i in range(5_000_000)]

sample_func()

##OUTPUT
% /usr/bin/python3 memory_profiler_sample.py
RAM used: 3.60 MB
```

Techniques for optimising memory usage

Effective data structure usage

Choosing the right data structure is one of the most important ways to reduce RAM usage and improve performance. Different structures store data differently and have different memory footprints.

Array vs list: Using a regular list to store many numbers is much less efficient in RAM than using an array object. More memory allocations occur in a list, each of which take time; calculations on larger objects are less cache-friendly and more RAM is used overall, making less of it available to other programs.